

SKRIPTA ZA UČENJE

Družbalica - backend tok za rezervacije

ASP.NET Core · Entity Framework Core · SQL Server · REST API · DTO · HTTP

6. avgust 2026.

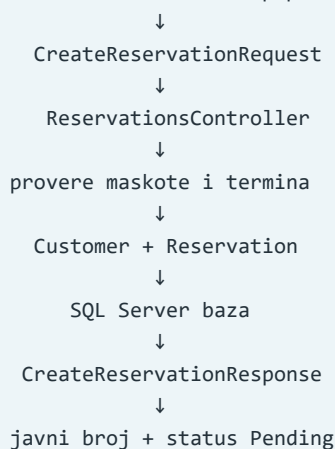
1. Šta smo danas napravili

Danas smo napravili backend deo javnog toka za iznajmljivanje maskote u aplikaciji Družbalica.

Korisnik još nema React formu, ali backend sada može da primi podatke buduće forme, proveri ih, sačuva klijenta i rezervaciju u SQL bazi i vrati javni broj zahteva.

TEXT

Korisnik izabere maskotu i popuni formu



Napravili smo:

- modele `Customer` i `Reservation`;
- enum `ReservationStatus`;
- veze rezervacije sa klijentom i maskotom;
- EF Core konfiguraciju i migraciju;
- request i response DTO klase;
- `POST /api/reservations` endpoint;
- validacije maskote i termina;
- generisanje javnog broja;

- čuvanje povezanih podataka;
- ručni test stvarnim HTTP zahtevom.

2. Najvažnija slika današnjeg gradiva

Svaki fajl ima svoju odgovornost:

Fajl	Uloga
<code>Customer.cs</code>	Entitet klijenta koji se čuva u bazi
<code>Reservation.cs</code>	Entitet rezervacije koji se čuva u bazi
<code>ReservationStatus.cs</code>	Dozvoljeni statusi rezervacije
<code>CreateReservationRequest.cs</code>	Podaci koje frontend sme da pošalje
<code>ReservationsController.cs</code>	Prima zahtev, proverava ga i čuva podatke
<code>CreateReservationResponse.cs</code>	Podaci koje API vraća korisniku
<code>ApplicationDbContext.cs</code>	Povezuje entitete sa tabelama i podešava veze
migracija	Opisuje konkretne promene SQL baze
<code>Program.cs</code>	Podešava aplikaciju i JSON prikaz enum vrednosti

Najvažnije razlike:

TEXT

Request = šta stiže spolja
 Entity = šta čuvamo u bazi
 Response = šta vraćamo spolja

Request i response nisu tabele. `Customer` i `Reservation` jesu entiteti koji postaju tabele.

3. Model Customer

`Customer` predstavlja osobu koja šalje zahtev za iznajmljivanje.

CSHARP

```
public class Customer
{
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    public string FirstName { get; set; } = string.Empty;

    [Required]
    [MaxLength(100)]
    public string LastName { get; set; } = string.Empty;

    [Required]
    [MaxLength(30)]
    [Phone]
    public string Phone { get; set; } = string.Empty;

    [Required]
    [MaxLength(100)]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;
}
```

Telefon je `string`, a ne broj, zato što može da sadrži znak `+`, razmake i vodeću nulu.

`[Phone]` i `[EmailAddress]` proveravaju format. Oni ne proveravaju da li broj ili email stvarno postoje.

4. Model Reservation

`Reservation` predstavlja zahtev za iznajmljivanje koji se trajno čuva u bazi.

Najvažnija svojstva:

- `Id` — interni primarni ključ;
- `PublicNumber` — broj koji dobija klijent;
- `MascotId` — strani ključ prema maskoti;
- `CustomerId` — strani ključ prema klijentu;
- `StartAt` i `EndAt` — početak i kraj termina;
- `EventLocation` — lokacija događaja;
- `Note` — opciona napomena;
- `Status` — trenutno stanje zahteva;
- `CreatedAt` — vreme kreiranja zahteva.

Koristimo `StartAt` i `EndAt` umesto odvojenih polja za datum i vreme. Tako je lakše porediti termine.

CSHARP

```
public string? Note { get; set; }
```

Znak `?` znači da napomena može biti `null`, odnosno korisnik ne mora da je unese.

5. Primarni i strani ključevi

Primarni ključ jedinstveno označava red u njegovoj tabeli:

TEXT

Customer.Id
Mascot.Id
Reservation.Id

Strani ključ povezuje red sa drugom tabelom:

TEXT

Reservation.CustomerId → Customer.Id
Reservation.MascotId → Mascot.Id

Primer:

TEXT

Customers
Id | FirstName
5 | Ana

Reservations
Id | CustomerId | MascotId
12 | 5 | 1

Rezervacija broj 12 pripada Ani zato što je njen `CustomerId` jednak Aninom `Id`.

Jedan klijent može imati više rezervacija. Jedna maskota takođe može imati više rezervacija kroz različite termine.

6. Navigaciona svojstva

U `Reservation` imamo i strani ključ i navigaciono svojstvo:

CSHARP

```
public int CustomerId { get; set; }  
public Customer Customer { get; set; } = null!;
```

- `CustomerId` je vrednost koja se čuva u koloni baze;
- `Customer` omogućava pristup celom povezanom objektu u C# kodu.

Zato kasnije možemo da napišemo:

CSHARP

```
reservation.Customer.FirstName
```

Oba naziva `Customer` počinju velikim slovom jer je prvi naziv klase, a drugi javno svojstvo. Javna svojstva u C# koriste PascalCase.

7. Enum ReservationStatus

Enum predstavlja ograničen skup dozvoljenih vrednosti:

CSHARP

```
public enum ReservationStatus
{
    Pending,
    Confirmed,
    Rejected,
    Cancelled,
    Completed
}
```

Prednost enuma je što status ne može biti proizvoljan tekst ili pogrešno napisana reč.

Novi javni zahtev dobija **Pending**, jer prvo čeka obradu zaposlenog.

U bazi status čuvamo kao tekst pomoću:

CSHARP

```
entity.Property(reservation => reservation.Status)
    .HasConversion<string>()
    .HasMaxLength(20);
```

Bez konverzije bi se enum podrazumevano čuvao kao broj, na primer **Pending** = 0.

8. EF Core veze

Vezu rezervacije sa klijentom konfigurisali smo ovako:

CSHARP

```
entity.HasOne(reservation => reservation.Customer)
    .WithMany()
    .HasForeignKey(reservation => reservation.CustomerId)
    .OnDelete(DeleteBehavior.Restrict);
```

Čitanje red po red:

- **HasOne** — rezervacija ima jednog klijenta;
- **WithMany** — klijent može imati više rezervacija;
- **HasForeignKey** — veza koristi **CustomerId**;
- **Restrict** — klijent se ne može obrisati dok ima rezervacije.

Istu vrstu veze napravili smo između rezervacije i maskote.

Restrict čuva poslovnu istoriju. Ne želimo da brisanje klijenta ili maskote automatski izbriše stare rezervacije.

9. Migracija

Nakon promene modela i **ApplicationDbContext** konfiguracije napravili smo migraciju:

POWERSHELL

```
Add-Migration AddCustomerAndReservation
```

Zatim smo je primenili:

POWERSHELL

```
Update-Database
```

Razlika:

- `Add-Migration` generiše C# opis promena baze;
- `Update-Database` izvršava te promene nad SQL Serverom.

Migracija je napravila tabele `Customers` i `Reservations`, strane ključeve i indekse.

Za `PublicNumber` smo dodali jedinstveni indeks:

CSHARP

```
entity.HasIndex(reservation => reservation.PublicNumber)
    .IsUnique();
```

Baza zato neće dozvoliti dve rezervacije sa istim javnim brojem.

10. Request DTO

`CreateReservationRequest` opisuje podatke koje frontend šalje endpointu.

Sadrži:

- `MascotId`;
- `FirstName` i `LastName`;
- `Phone` i `Email`;
- `StartAt` i `EndAt`;
- `EventLocation`;
- opcionu `Note`.

Ne sadrži:

- `Id` — generiše ga baza;
- `CustomerId` — klijent još nije sačuvan;
- `PublicNumber` — generiše ga backend;
- `Status` — određuje ga backend;
- `CreatedAt` — određuje ga backend.

Korisnik ne kuca `MascotId`. Frontend ga automatski uzima sa stranice izabrane maskote i šalje u pozadini.

11. Zašto endpoint ne prima `Reservation` direktno

Kada bi korisnik slao ceo entitet, mogao bi da pokuša da postavi interne vrednosti:

JSON

```
{
  "id": 500,
  "status": "confirmed",
  "createdAt": "2020-01-01"
}
```

To ne želimo. Korisnik ne sme sam da potvrdi rezervaciju niti da određuje interni identifikator i datum kreiranja.

DTO zato predstavlja bezbedan ugovor API-ja: dozvoljava samo polja koja korisnik zaista sme da pošalje.

12. Response DTO

Nakon uspešnog čuvanja API vraća samo podatke potrebne korisniku:

CSHARP

```
public class CreateReservationResponse
{
    public string PublicNumber { get; set; } = string.Empty;
    public ReservationStatus Status { get; set; }
}
```

Primer JSON odgovora:

JSON

```
{
  "publicNumber": "RES-20260806-9DE039D1",
  "status": "pending"
}
```

Ne vraćamo interni `Id`, `CustomerId` niti cele navigacione objekte.

13. Kontroler i dependency injection

Kontroler dobija `ApplicationDbContext` kroz konstruktor:

CSHARP

```
public ReservationsController(ApplicationDbContext context)
{
    _context = context;
}
```

ASP.NET Core pravi i prosleđuje kontekst. Kontroler ne koristi `new ApplicationDbContext(...)`.

Atributi određuju ponašanje i rutu:

CSHARP

```
[ApiController]
[Route("api/[controller]")]
public class ReservationsController : ControllerBase
```

Pošto se kontroler zove `ReservationsController`, ruta postaje:

TEXT

```
/api/reservations
```

`[HttpPost]` označava akciju koja kreira novi zahtev.

14. Validacije u endpointu

Prvo tražimo maskotu:

CSHARP

```
var mascot = await _context.Mascots.FindAsync(request.MascotId);
```

Zatim proveravamo:

- 1 da li maskota postoji;
- 2 da li je dostupna za iznajmljivanje;
- 3 da li je `EndAt` posle `StartAt`;
- 4 da li je početak termina u budućnosti.

Primer poređenja termina:

CSHARP

```
if (request.EndAt <= request.StartAt)
{
    return BadRequest(new
    {
        message = "Kraj termina mora biti posle početka termina."
    });
}
```

Ova provera je u kontroleru jer poredi dva različita svojstva. Običan atribut poput `[MaxLength]` proverava samo jedno svojstvo.

Važna poslovna pravila moraju biti na backendu. Frontend se može zaobići direktnim HTTP zahtevom.

15. Pretvaranje requesta u entitete

Podatke jedne forme razdvajamo na dva objekta.

CSHARP

```
var customer = new Customer
{
    FirstName = request.FirstName.Trim(),
    LastName = request.LastName.Trim(),
    Phone = request.Phone.Trim(),
    Email = request.Email.Trim()
};
```

`Trim()` uklanja slučajne razmake sa početka i kraja teksta.

Zatim pravimo rezervaciju i povezujemo je sa klijentom:

CSHARP

```
var reservation = new Reservation
{
    PublicNumber = publicNumber,
    MascotId = mascot.Id,
    Customer = customer,
    StartAt = request.StartAt,
    EndAt = request.EndAt,
    EventLocation = request.EventLocation.Trim(),
    Note = request.Note?.Trim(),
    Status = ReservationStatus.Pending,
    CreatedAt = DateTime.UtcNow
};
```

Klijent još nema `Id`, pa koristimo:

CSHARP

```
Customer = customer
```

Entity Framework razume vezu između objekata.

16. Čuvanje povezanih podataka

CSHARP

```
_context.Reservations.Add(reservation);  
await _context.SaveChangesAsync();
```

EF Core zatim:

TEXT

1. ubacuje Customer u tabelu Customers
2. SQL Server generiše Customer.Id
3. EF postavlja taj CustomerId na rezervaciju
4. ubacuje Reservation u tabelu Reservations

Jedan poziv `SaveChangesAsync()` čuva ceo povezani graf objekata.

`async` i `await` omogućavaju aplikaciji da ne blokira nit dok čeka bazu.

17. Javni broj zahteva

Javni broj generišemo na backendu:

CSHARP

```
var publicNumber =  
    $"RES-{DateTime.UtcNow:yyyyMMdd}-{Guid.NewGuid().ToString("N")[..8].ToUpperInvariant()}";
```

Primer:

TEXT

RES-20260806-9DE039D1

- RES označava rezervaciju;
- 20260806 je datum;
- poslednjih osam znakova je nasumični deo.

Klijent koristi javni broj umesto internog rednog `Id`.

18. HTTP statusni kodovi

Endpoint koristi:

Status	Značenje u našem endpointu
201 Created	Klijent i rezervacija su uspešno napravljeni
400 Bad Request	Maskota nije za najam ili termin nije ispravan
404 Not Found	Maskota sa poslatim <code>MascotId</code> ne postoji

Zbog `[ApiController]`, neispravan format request DTO-a takođe automatski može vratiti `400 Bad Request`.

19. JSON prikaz enum vrednosti

EF konverzija statusa u tekst važi za bazu, ali ne određuje automatski JSON odgovor.

Zato smo u `Program.cs` dodali `JsonStringEnumConverter`:

CSHARP

```
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(
            new JsonStringEnumConverter(JsonNamingPolicy.CamelCase));
    });
```

Zahvaljujući tome API vraća:

JSON

```
"status": "pending"
```

umesto:

JSON

```
"status": 0
```

20. Ručni test

Endpoint smo testirali stvarnim zahtevom za maskotu sa `Id = 1`.

JSON

```
{
  "mascotId": 1,
  "firstName": "Test",
  "lastName": "Korisnik",
  "phone": "+381641234567",
  "email": "test@example.com",
  "startAt": "2026-08-10T16:00:00Z",
  "endAt": "2026-08-10T19:00:00Z",
  "eventLocation": "Novi Sad",
  "note": "Proba rezervacije"
}
```

Dobijen je uspešan odgovor sa javnim brojem i statusom `pending`. To potvrđuje tok od HTTP zahteva do SQL baze i nazad.

21. Šta namerno još nismo uradili

Nismo još napravili:

- React formu za iznajmljivanje;
- interni pregled rezervacija;
- promenu statusa;
- prijavu zaposlenih;
- proveru preklapanja pri potvrđivanju rezervacije;
- traženje postojećeg klijenta po telefonu ili emailu.

Za sada svaki zahtev pravi novi `Customer` zapis. Kasnije možemo upozoravati na moguće duplikate.

Zahtevi sa istim terminom trenutno mogu imati status `Pending`. Proveru preklapanja obavezno radimo kada zaposleni pokuša da postavi status `Confirmed`.

22. Najčešće nedoumice

Da li request pravi tabelu?

Ne. Request je samo paket podataka koji stiže u API.

Da li korisnik kuca MascotId?

Ne. Frontend ga automatski šalje na osnovu otvorene maskote.

Zašto imamo i CustomerId i Customer?

CustomerId se čuva u bazi, a Customer omogućava rad sa celim objektom u C# kodu.

Zašto su Status i CreatedAt na kraju klase?

Samo zbog preglednosti. Redosled svojstava uglavnom ne utiče na rad aplikacije.

Zašto validaciju termina radimo u kontroleru?

Zato što poredi više polja i predstavlja poslovno pravilo backend toka.

Zašto ne pushujemo posle svake klase?

Commit je najkorisniji kada predstavlja jasnu, zaokruženu promenu koja može da se izgradi i objasni.

23. Pitanja za proveru znanja

- 1 Koja je razlika između entiteta, request DTO-a i response DTO-a?
- 2 Zašto korisnik ne sme da šalje Status i CreatedAt?
- 3 Šta povezuje Reservation i Customer u bazi?
- 4 Koja je razlika između CustomerId i Customer?
- 5 Šta znači .WithMany()?
- 6 Zašto koristimo DeleteBehavior.Restrict?
- 7 Koja je razlika između Add-Migration i Update-Database?
- 8 Zašto proveravamo da je EndAt posle StartAt?
- 9 Šta radi SaveChangesAsync()?
- 10 Zašto klijentu vraćamo PublicNumber, a ne interni Id?
- 11 Šta znači HTTP status 201 Created?
- 12 Zašto važna pravila ne smeju da postoje samo na frontendu?

24. Vežba bez gledanja u kod

Pokušaj svojim rečima da ispričaš ovaj tok:

TEXT

```
POST /api/reservations
→ request DTO
→ validacija
→ Customer
→ Reservation
→ SaveChangesAsync
→ response DTO
→ 201 Created
```

Ako možeš da objasniš ulogu svakog koraka bez pamćenja tačne sintakse, razumeš suštinu gradiva.

25. Kratak rezime

Danas nismo učili mnogo potpuno nove C# sintakse. Koristili smo poznate klase, svojstva, objekte i `if` uslove u stvarnom backend toku.

Novo gradivo su prvenstveno backend koncepti:

- povezani entiteti i strani ključevi;
- navigaciona svojstva;
- enum statusi;
- request i response DTO-i;
- POST endpoint;
- HTTP statusni kodovi;
- poslovna validacija;
- asinhrono čuvanje povezanih podataka;
- migracija za više povezanih tabela.

Sledeći korak je React forma koja će koristiti ovaj endpoint. Kada podaci iz forme prođu kroz ceo današnji tok, uloge svih klasa postaće još jasnije.